

MCI_ADV 사용자 매뉴얼

대량 재난 사고 시뮬레이션 플랫폼

버전 1.0 | 2026년 4월

목차

1. 개요
2. 시스템 요구사항
3. 설치 방법
4. 프로젝트 구조
5. 빠른 시작 가이드
6. 시나리오 생성
7. 시뮬레이션 엔진
8. 대시보드 (Streamlit)
9. 배치 실험 파이프라인
10. 정책 조합 (64개 시나리오)
11. 평가 지표
12. API 설정
13. 문제 해결
14. 부록

1. 개요

MCI_ADV는 대량 재난 사고(Mass Casualty Incident) 발생 시 **환자 이송 최적화**를 위한 시뮬레이션 및 분석 플랫폼입니다. 구급차(AMB)와 드론(UAV)을 활용한 64가지 배치 정책 조합을 평가하여 주어진 사고 지점에서의 최적 전략을 도출합니다.

핵심 기능

- 시나리오 생성:** 두 가지 도로 데이터 백엔드 지원
 - 카카오 모빌리티 API** (`is_use_time=True`) — 실시간/미래시간 교통정보 반영, 한국 한정 유료 키 필요
 - OSRM** (`is_use_time=False`) — 오픈소스 라우팅 엔진, 카카오 키 불필요. 정적 도로 그래프 기반 (distance/velocity로 시간 산출). 외부 리뷰어/공개 사용자를 위한 대안.
- 시뮬레이션 엔진:** 이산 사건 시뮬레이션(DES), 64개 규칙 조합 (완전요인설계)
- 통계 분석:** ANOVA (One-way / RCBD / Reduced Factorial), EMM 기반 사후검정, CLD (Piepho 2004)
- 웹 대시보드:** Streamlit 기반 인터랙티브 UI (지도, 분석, 데이터 편집)
- 배치 처리:** 수백~수천 개 좌표 대규모 실험

시스템 파이프라인

좌표 입력 → 시나리오 생성 → 시뮬레이션 (64규칙 x N샘플) → 분석 및 시각화

(위도, 경도)	카카오 API 또는 OSRM CSV/YAML/JSON 출력	이벤트 기반 시뮬 results_*.txt	ANOVA, 순위 Folium 지도
----------	-------------------------------------	----------------------------	------------------------

2. 시스템 요구사항

하드웨어

- CPU: 멀티코어 권장 (배치 실험 시)
- RAM: 최소 8GB, 대규모 실험 시 16GB 권장
- 저장공간: 1000개 좌표 실험당 약 500MB

소프트웨어

- Python 3.9 이상
- Windows 10/11, macOS, 또는 Linux
- 인터넷 연결 (도로 데이터 API 호출용 — 카카오 또는 OSRM)

API 키 / 라우팅 백엔드 (택1)

옵션 A — 카카오 모빌리티 API 사용 (`is_use_time=True`)

- **카카오 REST API 키** (아래 서비스 활성화 필요):
 - 카카오 모빌리티 (길찾기 API) — 도로 거리 + 실시간/미래 duration
 - 카카오 로컬 (키워드/주소 검색) — 좌표 검색 / 역지오코딩

옵션 B — OSRM 백엔드 사용 (`is_use_time=False`)

- 카카오 키 **불필요**. 외부 리뷰어/공개 사용자에게 권장.
- **OSRM** HTTP API로 도로 거리 + duration을 받는다.
- 기본값은 공식 데모 서버(<https://router.project-osrm.org>)지만 fair-use 정책이 있으므로 운영용은 자체 호스팅(도커) 권장. 자세한 절차는 프로젝트 루트 [README.md](#) "OSRM 백엔드" 섹션 참고.
- 이 모드에서는 좌표 검색/역지오코딩 UI가 동작하지 않을 수 있다(카카오 로컬 의존). CSV로 직접 좌표를 넣는 배치 모드는 정상 동작.

3. 설치 방법

3.1 저장소 클론

```
git clone <repository-url>
cd MCI_ADV
```

3.2 의존성 패키지 설치

```
pip install -r requirements.txt
```

주요 패키지:

- `streamlit` (≥ 1.34) - 웹 대시보드
- `folium`, `streamlit-folium` - 지도 렌더링
- `pandas`, `numpy` - 데이터 처리
- `scipy`, `statsmodels` - 통계 분석
- `pyyaml` - 설정 파일
- `requests` - API 호출
- `haversine` - 직선 거리 계산
- `openpyxl` - 엑셀 파일 I/O
- `geopandas`, `shapely` - 지리 경계 처리 (배치 실험)

3.3 필수 데이터 파일

아래 파일을 `scenarios/` 디렉토리에 배치하세요:

파일	설명	필수
<code>scenarios/안전센터와 소방서.csv</code>	소방서/119안전센터 마스터 데이터	필수
<code>scenarios/엑셀 결합 데이터.xlsx</code>	병원 마스터 (병상, 수술실, 헬기장, 등급)	필수
<code>scenarios/DISTANCE_MATRIX_FINAL.xlsx</code>	사전 계산 거리 행렬	선택

4. 프로젝트 구조

```

MCI_ADV/
├── src/
│   ├── sce_src/
│   │   ├── orchestrator.py          # 마스터 오케스트레이터 (생성+실행 통합)
│   │   ├── make_csv_yaml_dynamic.py # 시나리오 데이터 생성기 (API 호출, CSV/YAML)
│   │   └── BatchLab.py              # 레거시 배치 처리기
│   ├── sim_src/
│   │   ├── main.py                  # 진입점 (RunManager)
│   │   ├── ScenarioManager.py       # 설정 로드, 개체 초기화
│   │   ├── EntityManager.py         # 개체 상태 관리 (환자/병원/구급차/UAV)
│   │   ├── EventManager.py          # 이벤트 큐 및 시뮬레이션 루프
│   │   ├── RuleManager.py           # 64개 정책 규칙 정의
│   │   └── MCIEnvironment_gymnasium.py # Gymnasium 환경 래퍼
│   └── vis_src/
│       ├── MCI_Streamlit.py         # 메인 대시보드 (~5000 라인)
│       └── pages/
│           ├── Generate.py          # 시나리오 생성 UI
│           ├── ResultsCompare.py    # 다좌표 결과 비교
│           └── BatchExperiment.py   # 배치 실험 대시보드
├── scenarios/                       # 생성된 시나리오 데이터
├── results/                         # 시뮬레이션 결과
└── experiment_logs/                 # 실행 로그
  
```

```
└─ experiment_1/
└─ requirements.txt
```

배치 실험 파이프라인

5. 빠른 시작 가이드

5.1 단일 좌표 (대시보드)

1. 대시보드 실행:

```
streamlit run src/vis_src/MCI_Streamlit.py
```

2. **Generate** 페이지로 이동 (사이드바)

3. 설정 입력:

- 프로젝트 경로 (자동 감지)
- 카카오 REST API 키 (**is_use_time** 체크 시) 또는 OSRM URL (체크 해제 시)
- 출발 날짜/시간 (카카오 모드 전용)
- 사고 좌표 (검색 또는 직접 입력)

4. 파라미터 설정:

- 사고 규모 (환자 수): 기본값 30
- 구급차 수: 기본값 30
- UAV 수: 기본값 3
- 구급차 속도: 40 km/h
- UAV 속도: 80 km/h
- 환자 인계 시간: 구급차 10분 / UAV 15분

5. **"Generate & Run"** 클릭 → 시나리오 생성 + 시뮬레이션 자동 실행

6. 결과 확인 (메인 대시보드):

- **Scenarios** 탭: 이벤트 로그, 환자 타임라인
- **Maps** 탭: Folium 지도 위 경로 시각화
- **Analytics** 탭: ANOVA, 시나리오 순위

5.2 명령줄 인터페이스 (단일 좌표)

```
# 1단계: 시나리오 생성 - Kakao 모드
python src/sce_src/make_csv_yaml_dynamic.py \
  --base_path . \
  --latitude 37.5665 --longitude 126.9780 \
  --is_use_time true \
  --kakao_api_key YOUR_API_KEY \
  --departure_time 202604031400 \
  --incident_size 30 --amb_count 30 --uav_count 3
```

```
# 1단계 (대안): 시나리오 생성 - OSRM 모드 (카카오 키 불필요)
python src/sce_src/make_csv_yaml_dynamic.py \
  --base_path . \
  --latitude 37.5665 --longitude 126.9780 \
  --is_use_time false \
  --osrm_url http://localhost:5000 \
  --incident_size 30 --amb_count 30 --uav_count 3

# 2단계: 시뮬레이션 실행
python src/sim_src/main.py --config_path scenarios/exp_.../config_(lat,lon).yaml

# 2b단계 (선택): 환자별 트레이스 로깅과 함께 실행
python src/sim_src/main.py --config_path scenarios/exp_.../config_(lat,lon).yaml -
-trace
# → results/ 폴더에 trace_*.json 생성 (Scenarios 탭 Trace Replay에서 시각화)
```

6. 시나리오 생성

6.1 처리 흐름

```
입력 좌표 (위도, 경도)
↓
병원 마스터 데이터 로드 (엑셀 결합 데이터.xlsx)
↓
검색 반경 내 병원 필터링
├─ Tier 3 (상급종합병원): 종별코드 = 1
├─ Tier 2 (종합병원):      종별코드 = 11
└─ Tier 1 (병원/요양병원): 종별코드 = 21, 28, 29, 31
↓
카카오 모빌리티 API 또는 OSRM → 각 병원까지의 도로 거리 및 소요 시간
↓
소방서 데이터 로드 (안전센터와 소방서.csv)
↓
유클리드 거리 기준 최근접 30개 소방서 선택
↓
카카오 모빌리티 API 또는 OSRM → 각 소방서까지의 도로 거리 및 소요 시간
↓
CSV 파일 생성:
├─ patient_info.csv      (환자 중증도 분포: Green/Yellow/Red)
├─ hospital_info_road.csv (도로 거리 기준 병원 목록)
├─ hospital_info_euc.csv  (직선 거리 기준 병원 목록)
├─ amb_info_road.csv      (구급차 출동 정보, 도로)
├─ amb_info_euc.csv       (구급차 출동 정보, 직선)
├─ uav_info.csv           (UAV 출동 정보, 헬기장 병원만)
├─ distance_Hos2Hos_*.csv (병원-병원 거리 행렬)
└─ distance_Hos2Site_*.csv (병원-사고지점 거리 행렬)
↓
config_(lat,lon).yaml 생성
```

↓
경로 JSON 저장 (routes/center2site/ 및 routes/hos2site/)

6.2 주요 파라미터

파라미터	기본값	설명
<code>incident_size</code>	30	환자 수
<code>amb_count</code>	30	구급차 수
<code>uav_count</code>	3	UAV 수
<code>amb_velocity</code>	40	구급차 속도 (km/h)
<code>uav_velocity</code>	80	UAV 속도 (km/h)
<code>amb_handover_time</code>	10.0	구급차 환자 인계 시간 (분)
<code>uav_handover_time</code>	15.0	UAV 환자 인계 시간 (분)
<code>buffer_ratio</code>	1.5	병원 검색 반경 배율
<code>is_use_time</code>	True	True: 카카오 API duration 기반 / False: OSRM 정적 거리(distance/velocity) 기반. False 모드에서도 OSRM duration이 CSV에 함께 저장되므로, 동일 시나리오 폴더로 시물을 재실행할 때 YAML의 <code>is_use_time</code> 을 True로 바꾸면 OSRM duration 기반 시물이 가능합니다.
<code>osrm_url</code>	(env <code>MCI_OSRM_URL</code> 또는 <code>https://router.project-osrm.org</code>)	OSRM HTTP API base URL. <code>is_use_time=False</code> 일 때만 사용. 데모 서버는 fair-use 정책이 있으므로 자체 호스팅(도커) 권장.
<code>duration_coeff</code>	1.0	소요 시간 가중치 계수
<code>total_samples</code>	30	규칙당 시물레이션 반복 횟수
<code>random_seed</code>	0	재현성을 위한 랜덤 시드

6.3 환자 중증도 분포

환자는 아래 비율로 중증도가 무작위 배정됩니다:

중증도	비율	생존 모델	치료 가능 병원
Green (경증)	~60%	높은 생존율, 느린 감소	Tier 1, 2, 3
Yellow (중등증)	~20%	중간 생존율, 보통 감소	Tier 2, 3
Red (중증)	~20%	낮은 생존율, 급격 감소	Tier 3만 가능

6.4 병원 등급 체계

등급	한국 명칭	종별코드	수용 능력
Tier 3	상급종합병원	1	모든 중증도, 일부 헬기장 보유
Tier 2	종합병원	11	Green + Yellow만 치료
Tier 1	병원/요양병원	21, 28, 29, 31	Green만 치료

7. 시뮬레이션 엔진

7.1 아키텍처

이산 사건 시뮬레이션(DES) 방식으로, 우선순위 이벤트 큐를 사용합니다.

```

RunManager (main.py)
├── ScenarioManager → 설정 로드, 개체 초기화
│   ├── EntityManager → 환자/병원/구급차/UAV 상태 추적
│   └── EventManager → 이벤트 큐 관리 (heapq)
├── RuleManager → 64개 규칙 조합 생성
└── MCIEnvironment → Gymnasium 호환 환경 래퍼
  
```

7.2 이벤트 유형

이벤트	설명
<code>onset</code>	사고 발생, 환자 구조 필요
<code>p_rescue</code>	환자 구조 완료 (중증도 확인)
<code>amb_arrival_site</code>	구급차 사고 현장 도착
<code>amb_arrival_hospital</code>	구급차 병원 도착
<code>uav_arrival_site</code>	UAV 사고 현장 도착
<code>uav_arrival_hospital</code>	UAV 병원 도착
<code>treat_start</code>	병원에서 치료 시작
<code>treat_end</code>	치료 완료

7.3 시뮬레이션 루프

각 규칙(64개) x 각 샘플(N회 반복)에 대해:

1. `env.reset()` - 시나리오 초기화
2. `EventManager.run_next()`로 시간순 이벤트 처리
3. 규칙이 행동 선택: 어떤 환자를, 어떤 병원에, 어떤 이송수단으로
4. 경과 시간 기반 생존확률 계산
5. 지표 기록: Reward, Time, PDR, Reward_woG, PDR_woG

7.4 트레이스 로깅 (`--trace`)

`python main.py --config_path <config.yaml> --trace`로 실행하면 환자별 이벤트 트레이스를 기록합니다:

- 기록 이벤트: onset, rescue, transport_start, hospital_arrival, diversion, care_start, care_complete
- 출력: results 디렉토리에 `trace_*.json` 생성
- 시각화: Scenarios 탭 → Simulation Trace Replay (Gantt chart)

7.5 전원(Diversion) 규칙

병원 용량 초과 시 환자를 다른 병원으로 전원합니다:

- 구급차 환자: 환자 중증도에 맞는 가장 가까운 가용 병원으로 이송
- UAV + Red 환자: 헬기장이 있는 Tier 3 병원으로만 이송 가능

8. 대시보드 (Streamlit)

8.1 실행 방법

```
streamlit run src/vis_src/MCI_Streamlit.py
```

브라우저에서 `http://localhost:8501`로 자동 열립니다.

8.2 메인 페이지 탭

Settings (사이드바)

- 프로젝트 경로 선택
- 실험 ID 선택
- 좌표 선택
- 현재 좌표 위치를 보여주는 미니맵

Scenarios 탭

- **실험 로그 뷰어:** 좌표별 로그 필터링
- **환자 타임라인:** 구조시각, 이송수단, 병원, 도착시각, 치료완료 시각
- **이벤트 테이블:** 전체 이벤트 로그 (Rule/Iteration 필터 가능)
- **Patient Story Animation:** 환자별 상태 변화를 시간축 위에 색상 바로 시각화 (Waiting → Rescued → Transport → Hospital → Completed)
- **Simulation Trace Replay:** `--trace` 플래그로 실행한 시뮬레이션의 환자별 Gantt chart (`trace_*.json` 필요)
 - 환자별 구조 → 이송 → 병원 도착 → 치료 시작 → 완료까지 타임라인
 - 중증도별 색상 구분 (Red/Yellow/Green/Black)
 - 이벤트 요약 통계 (Rescues, Transports, Arrivals, Diversions, Completed)

Maps 탭

상단 라디오 버튼으로 두 가지 모드 전환:

Static Map 모드:

- **인터랙티브 Folium 지도:** 경로 시각화
- **구급차 경로:** 소방서→사고지점 (보라색), 사고지점→병원 (청록색)
- **UAV 경로:** 헬기장 병원→사고지점 (출동), 사고지점→병원 (이송)
- **교통 혼잡도 색상:** 원활(녹색), 보통(파란색), 서행(노란색), 정체(빨간색)
- **경로 정보 팝업:** 거리(km), 소요시간(분)
- 표시 경로 멀티셀렉트 (렌더링 성능을 위한 제한 가능)

Animation 모드 (시뮬레이션 로그 기반):

- 시뮬레이션 로그 파일 선택 (시나리오 생성 로그는 자동 제외)
- Rule/Iteration 선택
- **이모지 마커:** 🚑 AMB, 🚁 UAV, 🟡 대기 환자, 🏥 치료 중, ✅ 치료 완료
- **실제 도로 경로:** route JSON 폴리라인(Kakao/OSRM)을 따라 차량 이동
- **환자 탑승 표시:** 환자 이송 중 빨간 글로우 + 🧑 오버레이
- **이동 방향 반영:** 경도 변화에 따라 이모지 좌우 반전
- **환자 클릭 팝업:** 환자 마커 클릭 시 이송 수단, 병원명, ER 대기(핸드오버) 시간, 치료 시간, 총 체류 시간 팝업 표시
- **컨트롤 바** (지도 하단 외부): Play/Pause 버튼, Replay 버튼, 시간 슬라이더, 시간 표시
- 확대/축소/팬 지원 (재생 중에도 가능)
- 프레임 수 슬라이더 조절 (30~300 프레임)

Analytics 탭

7개 서브탭으로 구성: **RAW Data** | **STAT Summary** | **ANOVA Suite** | **Pareto Dominance** | **Bootstrap / Non-Parametric** | **Power Analysis** | **Export**

- **RAW Data:** 각 메트릭의 반복별 원시값 (Reward, Time, PDR, Reward w.o.G, PDR w.o.G)
- **STAT Summary:** 시나리오별 평균, 표준편차, 95% CI + 시나리오 순위 (Reward↓, PDR↑, Time↑ 정렬)
- **ANOVA Suite:**
 - **모형:** One-way ($value \sim C(rule)$), RCBD ($value \sim C(rule) + C(run)$), Reduced Factorial (주 효과 + 2원 교호작용 + 블록)
 - **CRN 가정:** RCBD 및 Factorial 모드는 Common Random Numbers(공통 난수) 전제 — 각 run 내 64개 rule이 동일 랜덤 시드를 공유하므로 **run**이 유효한 블록 변수
 - **사후검정:**
 - RCBD/Factorial: **EMM(추정 주변 평균)** 기반 pairwise t-test, 모형의 MS_residual을 풀링 오차로 사용, Holm 보정
 - One-way: Games-Howell (이분산 로버스트)
 - Fallback: Pairwise Welch t-test + Holm 보정 (pingouin 미설치 시)
 - **CLD(Compact Letter Display):** Piepho(2004) absorption 알고리즘 — 그룹에 다중 문자 부여 가능 (예: "ab"); 공유 문자가 있으면 유의차 없음
 - **효과크기:** η^2 (에타제곱) 및 ω^2 (오메가제곱, 편향 보정)
 - **잔차 진단:** Shapiro-Wilk + Anderson-Darling 정규성, QQ plot, 히스토그램, Residuals vs Fitted
 - **등분산 검정:** Levene (Brown-Forsythe variant, center=median)
 - **RCBD 가법성:** Tukey 1-df 비가법성 검정
 - **A그룹 교집합:** Reward↑ $\cap$ Time↓ $\cap$ PDR↓ 세 지표 모두에서 최상위 CLD 그룹(문자 'a' 포함) 시나리오 추천
- **Pareto Dominance** (다목적 최적화 분석):
 - 통계적 Pareto 효율성: CLD 결과 기반 rule 간 지배 관계 판별
 - 비지배 정렬 (Pareto Layer): Layer 0 = 최적 프론트

- 3D 산점도 (Reward × Time × PDR), Layer별 색상 구분
- Dominance 카운트 테이블 (각 rule이 지배/피지배하는 rule 수)
- **Bootstrap / Non-Parametric** (정규성 위반 시 대안):
 - BCa Bootstrap CI ([scipy.stats.bootstrap](#)): 편향 보정 가속 신뢰구간
 - Friedman Test: RCBD 비모수 대안 + Conover 사후검정
 - Kruskal-Wallis: One-way 비모수 대안 + Dunn 사후검정
- **Power Analysis** (검정력 분석):
 - 사후 검정력: 관측된 효과크기(η^2)와 MSE 기반
 - 사전 표본 크기 권장: 목표 검정력(0.8)에 필요한 n 계산
 - 검정력 곡선 그래프 (n vs power)
- **Export** (출판용 내보내기):
 - ANOVA 테이블 LaTeX 형식 (APA 스타일)
 - CLD 결과 LaTeX 테이블
 - 전체 분석 번들 다운로드 (.txt)

Data Tables 탭

- 시나리오 CSV 파일 직접 편집
- 수정 전 자동 백업
- 수정된 데이터로 재실행 가능

Rerun 탭

- 기존 YAML 설정으로 시뮬레이션 재실행
- 시나리오 데이터 편집 후 유용

8.3 추가 페이지

Generate 페이지 ([pages/Generate.py](#))

- 전체 시나리오 생성 워크플로우
- 카카오 API 좌표 검색 (키워드 + 주소)
- 배치 좌표 입력 (다수 지점)
- 파라미터 프리셋 관리
- 생성 + 시뮬레이션 원클릭 실행

Results Compare ([pages/ResultsCompare.py](#))

- 다수 좌표 간 결과 비교
- 메트릭 나란히 비교
- Composite Score 및 Tier 분류 (A/B/C)
- **Cross-Scenario Meta-Analysis:**
 - Kendall's W 일치도 계수 (위치 간 순위 일치도)
 - Forest Plot (rule별 평균 ± CI across locations)
 - Stability Index (rule이 상위 유지되는 비율)
 - Rule × Location 교호작용 검정 (이원 ANOVA)

Batch Experiment ([pages/BatchExperiment.py](#))

- 5단계 워크플로우: 좌표 생성 → 확인 → 실행 → 진행 현황 → 시각화
- experiment_1 스크립트 함수 직접 import 재사용
- 중지/재개 기능이 있는 진행 추적
- **OSRM 모드 자동 감지**: 폴더명이 `_osrm`으로 끝나면 Kakao API 키 입력란·일일 한도·호출 추정치 UI를 자동으로 숨기고, API 예산 제한 없이 전체 좌표를 처리한다. Kakao 관련 설정은 `_dep_` 접미사 폴더에서만 표시된다.

9. 배치 실험 파이프라인

9.1 개요

`experiment_1/` 디렉토리에는 다수 좌표에 대한 대규모 실험용 스크립트가 포함되어 있습니다.

9.2 단계별 가이드

1단계: 랜덤 좌표 생성

```
python experiment_1/generate_coords.py \
  --n 1000 --seed 42 \
  --shp experiment_1/ctprvn.shp \
  --out experiment_1/coords_korea.csv
```

- 한국 육지 경계 내 N개 랜덤 좌표 생성
- Shapefile 기반 정밀 경계 판별
- `coord_id`, `lat`, `lon` 형식 CSV 및 미리보기 HTML 지도 출력

2단계: 배치 처리 실행

Kakao 모드 (실시간 교통정보)

```
python experiment_1/batch_runner.py \
  --coords experiment_1/coords_korea.csv \
  --kakao-api-key YOUR_API_KEY \
  --experiment-id exp_korea_random_1000 \
  --departure-time 202603311400 \
  --daily-limit 4900 \
  --total-samples 30
```

OSRM 모드 (오픈소스, 카카오 키 불필요)

```
python experiment_1/batch_runner.py \
  --coords experiment_1/coords_korea.csv \
  --is-use-time false \
  --osrm-url http://localhost:5000 \
  --experiment-id exp_korea_random_1000_osrm \
  --total-samples 30
```

주요 기능:

- **진행 추적:** `progress.json`에 각 좌표 처리 후 상태 저장
- **재개 기능:** 재시작 시 완료된 좌표 자동 건너뛰기
- **API 할당량 관리:** 일일 API 호출 추적, 한도 도달 시 일시 정지
- **오류 처리:** 실패 좌표 `--max-retries`로 재시도 가능

3단계: 결과 시각화

```
python experiment_1/visualize_coords.py \
  --coords experiment_1/coords_korea.csv \
  --progress experiment_1/progress.json \
  --clip-pct 2 --outlier-n 5
```

출력물 (`scenarios/{experiment_id}/` 폴더에 저장):

- `coords_map.html`: 인터랙티브 결과 지도 (Reward/Time/PDR 전환, OpenStreetMap/CartoDB 타일 전환)
- `coords_map_hist.pdf/png`: 주요 지표 분포 히스토그램
- `coords_map_rule_heatmap.pdf/png`: 64개 규칙 성능 히트맵 (3지표 × 4패널, GnBu 단조톤 컬러맵, 낮을수록 좋은 지표는 색상 반전)
- `coords_map_rule_effects.pdf/png`: 요인별 주효과 그래프 (ANOVA η^2 effect size, ★ Best level 표시, teal 단조톤 바)

9.3 Progress JSON 구조

```
{
  "experiment_id": "exp_korea_random_1000",
  "total": 1000,
  "statuses": {
    "0": {
      "status": "done",
      "sim_ok": true,
      "coord": "(lat,lon)",
      "api_calls": 45,
      "error": null
    }
  },
  "api_log": {
    "2026-03-31": 2450,
    "2026-04-01": 2400
  }
}
```

```
}  
}
```

10. 정책 조합 (64개 시나리오)

시뮬레이션은 완전요인설계(Full Factorial Design)로 **64개 규칙 조합**을 평가합니다:

요인

요인	수 준 수	내부 값	표시 이름
Patient Prioritization (환자 우선 순위)	2	START, ReSTART	START (초기 배차), ReSTART (재평가)
Hospital Selection (병원 선택)	2	RedOnly, YellowNearest	RedOnly (Red 환자 우선), YellowNearest (최근접 가용)
Transport Mode Selection — Red (이송 수단 선택 — Red)	4	OnlyUAV, Both_UAVFirst, Both_AMBFirst, OnlyAMB	UAV-only, UAV-first, AMB-first, AMB-only
Transport Mode Selection — Yellow (이송 수단 선택 — Yellow)	4	OnlyUAV, Both_UAVFirst, Both_AMBFirst, OnlyAMB	UAV-only, UAV-first, AMB-first, AMB-only

합계: 2 x 2 x 4 x 4 = **64개 조합**

규칙 명명 규칙

```
{Phase}, {병원선택}, Red {Red행동}, Yellow {Yellow행동}
```

예: START, RedOnly, Red Both_UAVFirst, Yellow OnlyAMB

11. 평가 지표

지표	설명	방향
Reward	전체 환자의 생존확률 합	높을수록 좋음
Time	전체 시뮬레이션 소요 시간 (분)	낮을수록 좋음
PDR	환자 사망률 (Patient Death Rate)	낮을수록 좋음
Reward w.o.G	Green 환자 제외 Reward	높을수록 좋음
PDR w.o.G	Green 환자 제외 PDR	낮을수록 좋음

생존확률 모델

각 환자의 생존확률은 시간 경과에 따라 중증도별로 감소합니다:

- **Green:** 느린 감소 (높은 기저 생존율)
- **Yellow:** 보통 감소
- **Red:** 급격한 감소 (시간에 민감한 중증 환자)

12. API 설정

12.1 카카오 REST API 키 발급 (`is_use_time=True` 모드)

1. [카카오 개발자](#) 사이트 방문
2. 애플리케이션 생성
3. 서비스 활성화:
 - **카카오 모빌리티** (길찾기 API)
 - **카카오 로컬** (검색 API)
4. **REST API 키** 복사

12.2 사용되는 API 엔드포인트 (Kakao 모드)

엔드포인트	용도	좌표당 호출 수
모빌리티 길찾기	병원까지 도로 거리/소요시간	~20-50회
모빌리티 길찾기	소방서까지 도로 거리/소요시간	~30회
로컬 키워드 검색	좌표 검색 (대시보드 전용)	검색당 1회
로컬 주소 검색	주소 조회 (대시보드 전용)	검색당 1회

12.3 카카오 API 할당량 관리

- 카카오 무료 티어: **일 5,000회**
- 좌표당 약 **50-80회** API 호출 필요
- `batch_runner.py`가 `api_log`를 통해 일일 사용량 추적
- `--daily-limit` 설정 (기본값: 4900)으로 안전 마진 확보

12.4 OSRM 백엔드 (`is_use_time=False` 모드)

카카오 키가 없거나 외부 공개 환경에서는 OSRM(오픈소스 라우팅 엔진)을 사용한다.

기본 동작: 환경변수 `MCI_OSRM_URL`이 설정되어 있으면 그 값을, 아니면 공식 데모 서버

<https://router.project-osrm.org>를 사용한다. CLI/UI에서 `--osrm_url`로 명시 오버라이드 가능.

자체 호스팅 (운영 권장):

```
# 한국 OSM 추출본 다운로드 + 사전처리
wget https://download.geofabrik.de/asia/south-korea-latest.osm.pbf
docker run -t -v "$(pwd):/data" osrm/osrm-backend osrm-extract -p /opt/car.lua /data/south-korea-latest.osm.pbf
docker run -t -v "$(pwd):/data" osrm/osrm-backend osrm-partition /data/south-
```

```
korea-latest.osrm
docker run -t -v "$(pwd):/data" osrm/osrm-backend osrm-customize /data/south-
korea-latest.osrm

# 라우팅 서버 기동
docker run -t -i -p 5000:5000 -v "$(pwd):/data" osrm/osrm-backend \
  osrm-routed --algorithm mld /data/south-korea-latest.osrm

# 사용
export MCI_OSRM_URL=http://localhost:5000
```

호출되는 OSRM 엔드포인트:

엔드포인트	용도
<code>/route/v1/driving/{lon1},{lat1};{lon2},{lat2}</code>	병원/소방서까지의 도로 distance + duration + GeoJSON 폴리라인

자체 호스팅 OSRM은 호출 한도가 없고(서버 자원 한계 내), `--daily-limit` 설정은 무의미하지만 호환을 위해 그대로 유지된다.

제한사항:

- 실시간 교통정보 없음 (정적 도로 그래프 기반)
- 도로 혼잡도 데이터 없음 → 대시보드 지도는 단일 색 폴리라인
- 좌표 검색/역지오코딩 UI는 카카오 로컬 의존이라 OSRM 모드에서 동작하지 않음. CSV 직접 입력 배치 모드 권장.

13. 문제 해결

자주 발생하는 문제

문제	원인	해결 방법
시나리오 생성 <code>RuntimeError</code>	사고지점 자체에 도로 없음 (rc=102): 산, 바다, 무인도	버그가 아닌 정상 동작. 1000개 좌표 배치 실험 기준 223개가 이 유형. 소방서 경로 문제는 0건.
<code>Exception: Impossible to divert</code>	코드 로직 버그: UAV+Red 환자는 헬기장+Tier3 병원에만 이송 가능. 유일한 해당 병원(예: 원광대, 용량16)이 가득 차면 비헬기장 Tier3 병원(전북대, 용량19)에 빈자리가 있어도 전원 실패.	알려진 엣지 케이스 (1000개 중 5개 영향). 병상 부족 아님 — 전체 Tier3 용량 (35)이 환자 수(30)보다 충분. <code>diversion_rule()</code> 에 fallback 로직 추가 필요.
API 401 Unauthorized	잘못된 카카오 API 키	카카오 개발자 콘솔에서 키 확인. 키가 없다면 <code>is_use_time=false</code> 로 OSRM 백엔드 사용

문제	원인	해결 방법
API 429 Rate Limit	카카오 일일 할당량 초과	24시간 대기, 할당량 증가, 또는 OSRM 백엔드로 전환
RuntimeError: 카카오 API 키가 없습니다 (is_use_time=True 모드)	is_use_time=True로 시나리오 생성하면서 --kakao_api_key 미지정	키 제공하거나 is_use_time=false로 OSRM 사용
OSRM 경로 없음 (code=NoRoute)	OSRM 그래프에 연결되지 않은 좌표 (해상/도서)	해당 좌표 제외. 카카오 102와 동일한 의미.
UnicodeEncodeError: cp949	Windows 콘솔 인코딩	PYTHONIOENCODING=utf-8 환경변수 설정
YAML 키 순서 크래시	yaml.dump의 sort_keys=True	sort_keys=False 사용 (수정 완료)
시뮬레이션 무한 대기	stdout 파이프 버퍼 가득 참	Popen + 데몬 스레드로 수정 완료

성능 팁

- 배치 실험: API 할당량 극대화를 위해 야간 실행 권장
- 대시보드: Analytics "Load Analysis Data" 버튼 사용 (지연 로딩)
- 지도: 표시 경로를 10-15개로 제한하면 렌더링 원활

부록

A. Config YAML 구조

```
entity_info:
  departure_time: "202604031400"
  patient:
    info_path: ./scenarios/exp_.../patient_info.csv
    total: 30
  hospital:
    info_road_path: ./scenarios/exp_.../hospital_info_road.csv
    info_euc_path: ./scenarios/exp_.../hospital_info_euc.csv
    distance_h2h_road_path: ./scenarios/exp_.../distance_Hos2Hos_road.csv
    distance_h2s_road_path: ./scenarios/exp_.../distance_Hos2Site_road.csv
  ambulance:
    info_road_path: ./scenarios/exp_.../amb_info_road.csv
    info_euc_path: ./scenarios/exp_.../amb_info_euc.csv
    velocity: 40
    handover_time: 10.0
  uav:
    info_path: ./scenarios/exp_.../uav_info.csv
    velocity: 80
    handover_time: 15.0

output_path: ./results/exp_.../
```

```
totalSamples: 30
randomSeed: 0
```

중요: `entity_info`의 키 순서는 반드시 `departure_time` → `patient` → `hospital` → `ambulance` → `uav`여야 합니다. 이 순서가 바뀌면 `ScenarioManager`가 dict 삽입 순서로 개체를 처리하기 때문에 크래시가 발생합니다.

B. 결과 파일 형식

RAW (`results_(lat,lon).txt`):

```
START, RedOnly, Red OnlyUAV, Yellow OnlyUAV 12.5 13.2 11.8 ...
START, RedOnly, Red OnlyUAV, Yellow Both_UAVFirst 14.1 12.9 ...
...
```

각 줄: 규칙 라벨 + N개 샘플 값. 5개 메트릭 블록(Reward, Time, PDR, Reward_woG, PDR_woG), 각각 64줄.

STAT (`results_(lat,lon)_stat.txt`):

```
START, RedOnly, Red OnlyUAV, Yellow OnlyUAV 12.50 0.72 0.26
```

각 줄: 규칙 라벨 + 평균, 표준편차, 95% 신뢰구간 반폭.

C. 소방서 데이터 형식

안전센터와 소방서 `.csv` (CP949 인코딩):

```
기관명, 주소, 전화번호, x좌표, y좌표, 관할구역, 대수
강남소방서, 서울특별시 강남구..., 02-3400-1119, 127.0495, 37.4979, ..., 5
```

D. 병원 엑셀 형식

엑셀 결합 데이터 `.xlsx`:

- 주요 컬럼: `요양기관명`, `종별코드`, `수술실수`, `병상수`, `헬기장 여부`, `x좌표`, `y좌표`, `주소`, `전화번호`

MCI_ADV - 대량 재난 사고 시뮬레이션 플랫폼
문의 사항은 [GitHub 저장소](#)를 방문해 주세요.